

SIMATS ENGINEERING
ASSESSMENT TOOL 3: Reverse Engineering Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

COURSE OUTCOME COVERED

CO4: *Analyze big data characteristics and challenges across domains including security and application aspects.*
(BL4)

Dave's Taxonomy: Articulation (Level 4)
Question Count: 5

Total Marks: 30

Code of Conduct

I BRIJESH.T/(192521391) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:  _____

Assessment Tasks

Reverse Engineering Task

For each task, study the described big data solution, infer how it was built, identify the underlying components, and explain what design decisions were likely made.

1. A big data platform collects billions of website clicks, mobile interactions, and customer transactions every day. Reverse engineer the probable distributed storage system, ingestion framework, and batch-processing tools used. Infer how structured and semi-structured data are separated and indexed for analysis. Identify the likely stream-processing engines supporting real-time analytics and reporting. Explain the scalability and partitioning strategies probably implemented for handling massive workloads. Describe how dashboards and business intelligence systems are likely connected to the analytics layer.
2. An e-commerce platform tracks cart additions, abandoned checkouts, product views, and payment behavior across regions. Reverse engineer the likely event streaming tools, customer profiling databases, and recommendation workflow used. Infer how session data, clickstream logs, and transaction records are synchronized for real-time personalization. Identify probable caching systems, distributed query engines, and machine learning stages involved. Explain how structured order data and semi-structured browsing data are merged for analytics. Deduce the scalability and fault-tolerance decisions likely implemented. Describe how dashboards and KPI monitoring are probably generated from the pipeline.
3. A big data healthcare system processes patient histories, diagnostic images, wearable sensor streams, and prescription records. Reverse engineer the probable hybrid database architecture managing structured and unstructured healthcare information. Infer the likely data integration tools connecting hospitals, laboratories, and insurance systems. Identify the streaming and machine learning frameworks probably used for disease prediction and monitoring. Explain how privacy, encryption, and compliance requirements are likely enforced in the platform. Describe the analytics pipeline that may support clinical decision-making and population health studies.

4. A smart city platform gathers traffic sensor feeds, CCTV metadata, weather updates, and public transport activity continuously. Reverse engineer the probable edge computing setup, distributed messaging system, and centralized storage design. Infer how geospatial analytics and streaming computations are handled for congestion prediction. Identify the likely databases used for time-series and location-aware queries. Explain how batch and real-time analytics are combined for urban planning insights. Deduce the security and access-control measures likely protecting citizen and infrastructure data. Describe how visualization systems probably deliver live operational intelligence.
5. A healthcare analytics system integrates patient records, wearable device readings, lab reports, and appointment histories. Reverse engineer the probable hybrid storage architecture handling sensitive structured and unstructured medical data. Infer the likely ETL pipelines and interoperability standards connecting hospitals and clinics. Identify how real-time monitoring and predictive health analytics are probably implemented. Explain the role of distributed processing frameworks in population-level analysis. Deduce the encryption, compliance, and audit mechanisms likely enforced for data governance. Describe how machine learning models may support diagnosis risk scoring and treatment recommendations.

Reverse Engineering Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Inference Accuracy	25%	Infers system components and flow with high accuracy	Mostly accurate inference	Basic inference	Weak inference
Understanding of Architecture	25%	Shows clear understanding of underlying big data architecture	Good understanding	Partial understanding	Poor understanding
Use of Technical Evidence	20%	Uses strong reasoning and technical evidence	Reasonable evidence	Limited evidence	No meaningful evidence
Depth of Analysis	15%	Analysis is deep and insightful	Reasonably deep	Basic depth	Superficial analysis
Clarity	15%	Clear and organized explanation	Generally clear	Somewhat unclear	Poorly presented

1. Big Data Platform for Website Clicks and Transactions

A platform handling billions of clicks, mobile interactions, and transactions would most likely use a **distributed big data architecture**.

- **Distributed Storage:** Hadoop HDFS or cloud storage such as Amazon S3 is likely used to store huge amounts of data across many servers.
- **Data Ingestion:** Apache Kafka can collect website clicks, mobile events, and transaction data continuously.
- **Batch Processing:** Apache Spark is likely used to process large historical datasets.
- **Structured Data:** Customer details, orders, and payments can be stored in relational databases such as MySQL or distributed databases.
- **Semi-structured Data:** Click logs, JSON files, and application events can be stored in a data lake.
- **Indexing:** Apache Hive, Elasticsearch, or similar systems can be used to organize and search large datasets.
- **Stream Processing:** Apache Spark Streaming or Apache Flink can process clicks and transactions in near real time.
- **Scalability:** Data is divided into partitions and stored across multiple machines. More servers can be added when the workload increases.
- **Fault Tolerance:** Replication allows another server to provide the data if one server fails.
- **Analytics:** Spark and SQL-based tools analyze customer behavior, sales, and trends.
- **Dashboards:** Tableau, Power BI, or similar BI tools can connect to the analytics layer and display sales, customer activity, and KPI reports.

Probable flow:

Website/Mobile/Transactions → Kafka → HDFS/S3 → Spark → Analytics/ML → BI Dashboards

2. E-Commerce Personalization Platform

An e-commerce company needs fast processing because customer actions must be analyzed while the customer is still using the website.

- **Event Streaming:** Apache Kafka is likely used to collect product views, cart additions, payments, and checkout events.
- **Customer Profiles:** Databases such as Cassandra, MongoDB, or Redis can store customer profiles and frequently accessed information.
- **Session Data:** Current customer sessions can be stored in Redis or another fast key-value database.
- **Clickstream Data:** Browsing events can be stored in a data lake such as HDFS or Amazon S3.
- **Transaction Data:** Orders and payments are stored in structured databases.
- **Real-Time Processing:** Apache Flink or Spark Streaming can process customer events immediately.
- **Caching:** Redis can provide fast access to product information, customer preferences, and recommendations.
- **Query Processing:** Spark SQL, Trino, or Presto can be used for distributed queries.
- **Machine Learning:** ML models can analyze previous purchases and browsing behavior to recommend products.
- **Data Merging:** Customer IDs, session IDs, and order IDs can be used to connect browsing data with structured transaction records.
- **Scalability:** Kafka partitions allow many consumers to process events at the same time.
- **Fault Tolerance:** Data replication and backup prevent data loss when a server fails.
- **Dashboards:** Power BI or Tableau can show conversion rate, abandoned carts, sales, customer activity, and other KPIs.

Probable flow:

Customer Events → Kafka → Flink/Spark → Customer Profile/ML → Recommendations → Website

3. Big Data Healthcare System

Healthcare systems handle different types of information, so a **hybrid database architecture** is likely required.

- **Structured Data:** Patient IDs, prescriptions, appointments, and laboratory results can be stored in relational databases.
- **Unstructured Data:** Diagnostic images, medical documents, and reports can be stored in a data lake or object storage.
- **Hybrid Storage:** HDFS/S3 can store large files, while databases such as PostgreSQL, MongoDB, or Cassandra can manage other healthcare records.
- **Data Integration:** Apache NiFi, Kafka, or ETL tools can transfer data between hospitals, laboratories, pharmacies, and insurance systems.
- **Interoperability:** Healthcare standards such as **HL7 and FHIR** can help different systems exchange information.
- **Stream Processing:** Kafka with Spark Streaming or Flink can process wearable sensor data continuously.
- **Machine Learning:** ML models can be used for disease prediction, patient risk scoring, and health monitoring.
- **Distributed Processing:** Spark can process large patient datasets for population-level studies.
- **Privacy:** Only authorized users should access patient information.
- **Encryption:** Data should be encrypted both during transmission and while stored.
- **Compliance:** Access control, auditing, data retention, and privacy policies help meet healthcare regulations.
- **Clinical Analytics:** Patient history, laboratory results, sensor data, and other information can be combined to support doctors' decisions.

Probable flow:

Hospitals/Labs/Wearables → Integration → Secure Data Lake + Databases → Spark/Flink → ML/Analytics → Clinical Dashboard

4. Smart City Big Data Platform

A smart city produces continuous data from sensors, cameras, transport systems, and weather stations.

- **Edge Computing:** Small edge devices near traffic sensors and cameras can process data locally before sending it to the central system.
- **Messaging:** Apache Kafka or MQTT can collect continuous sensor and transport events.
- **Central Storage:** HDFS, Amazon S3, or another distributed data lake can store historical data.
- **Real-Time Processing:** Apache Flink or Spark Streaming can analyze traffic and transport data immediately.
- **Geospatial Analytics:** Location information can be processed using tools such as PostGIS or GeoMesa.
- **Time-Series Database:** InfluxDB or TimescaleDB can store traffic, weather, and sensor measurements over time.
- **Congestion Prediction:** Machine learning models can use previous traffic patterns, weather, and current sensor readings to predict congestion.
- **Batch Processing:** Spark can analyze months or years of data for city planning.
- **Real-Time Analytics:** Streaming systems can identify traffic problems and send immediate alerts.
- **Security:** Authentication, encryption, firewalls, and network segmentation protect city infrastructure.
- **Access Control:** Different departments should receive access only to the data they need.
- **Visualization:** Power BI, Tableau, Grafana, or similar tools can display live traffic conditions and operational KPIs.

Probable flow:

Sensors/CCTV/Weather/Transport → Edge → Kafka/MQTT → Flink/Spark → Time-Series + Data Lake → Analytics → Live Dashboard

5. Healthcare Analytics and Predictive Health System

This system combines medical records with wearable and laboratory data, so it would probably use both structured and unstructured storage.

- **Structured Storage:** Patient records, appointments, prescriptions, and lab results can be stored in relational databases.
- **Unstructured Storage:** Medical reports, documents, and other large files can be stored in a data lake such as HDFS/S3.
- **Wearable Data:** Continuous sensor readings can be collected through Kafka or similar streaming systems.
- **ETL Pipelines:** Apache NiFi, Talend, or Spark can extract, clean, transform, and load healthcare data.
- **Interoperability:** HL7 and FHIR standards can help hospitals and clinics exchange patient information.
- **Real-Time Monitoring:** Kafka with Flink or Spark Streaming can analyze wearable readings and generate alerts.
- **Distributed Processing:** Apache Spark can process millions of patient records for population health analysis.
- **Machine Learning:** ML models can predict disease risk, identify high-risk patients, and support treatment decisions.
- **Encryption:** Patient data should be encrypted during storage and transmission.
- **Access Control:** Role-based access ensures that only authorized doctors, nurses, and administrators can access sensitive data.
- **Auditing:** Every access or modification of patient data can be recorded in audit logs.
- **Compliance:** Data governance policies should ensure privacy, secure storage, proper retention, and regulatory compliance.
- **Analytics:** Historical and real-time data can be combined to provide risk scores and support healthcare decision-making.

Probable flow:

Patient Records + Wearables + Labs + Appointments → ETL/Streaming → Secure Hybrid Storage → Spark/Flink → ML Models → Risk Scores & Recommendations → Healthcare Dashboard